

Enhancing GLM-Based Lapse Rate Models Using MID - Basic Tutorial

Reiji Kozuma^{*} Ryoichi Asashiba[†] Yoshihiro Matsumori[‡] Hirokazu Iwasawa[§]

2026-09-25

Abstract

This document presents ready-to-use code examples illustrating the foundational workflow proposed in the original paper (Kozuma et al., 2026) for enhancing GLM-based lapse rate models for EV calculation using a novel interpretation method, MID. We cover dataset preprocessing, model construction, and visualization techniques using the `midr` package.

Detailed theoretical explanations of the predictive models, evaluation metrics, and interpretation methods are left to the original paper; this document focuses on demonstrating practical usage through concrete R implementations. Separate documents provide the simplified EV valuation framework and the complete code required to replicate all empirical results in the paper.

Much of the implementation is designed to reproduce the research setup faithfully, which may include code structures that appear complex to beginners. Readers who wish to focus on the core methodology may initially execute the surrounding code as provided and focus their attention on the model refinement process in Section 2.3.2.

Table of Contents

1 Preparation

- 1.1 Loading Packages
- 1.2 Loading and Preprocessing Data
- 1.3 Evaluation Metric Functions

2 Model Construction

- 2.1 Reduced Baseline GLM
- 2.2 XGBoost Model
- 2.3 Baseline GLM / Improved GLM

3 Statistical Predictive Performance

- 3.1 Reduced Baseline GLM
- 3.2 XGBoost Model

^{*} Daido Life Insurance Company, rktkdm@gmail.com

[†] Daido Life Insurance Company, ryoichi.asashiba@gmail.com

[‡] Sompo Japan Insurance Inc., ymatsumori1@sompo-japan.co.jp

[§] Graduate School of Accountancy, Waseda University, iwahiro@bb.mbn.or.jp

3.3 Baseline GLM

3.4 Improved GLM

Acknowledgments

References

1 Preparation

1.1 Loading Packages

Load the R packages required for this tutorial.

```
library(midr)
library(tidyverse)
library(xgboost)
library(ROCR)
```

1.2 Loading and Preprocessing Data

First, we load the dataset from the SOA 10-Year Level Premium Term Lapse Study ([Society of Actuaries, 2014](#)) utilized in our research ^{*1}.

```
# install.packages("devtools")
# devtools::install_github("kasaai/insurance")
library(insurance)
df_raw <- lapse_study
```

Next, we preprocess the data for analysis. Aside from adding an ID column for tracking, the transformations follow the definitions described in the original paper.

```
# Add ID column
df_raw <- df_raw %>% mutate(ID = as.integer(rownames(df_raw)))

# Convert lapse_study_year from string "yyyy-(yyyy+1)" to integer yyyy,
# and format issue_year and duration
df_raw <- df_raw %>% mutate(
  lapse_study_year = as.integer(substr(lapse_study_year, 1, 4)),
  issue_year =
    if_else(issue_year == "Pre-1991", "-1990", issue_year) %>%
    as.ordered(),
  duration = case_when(
    duration == "6-9" ~ "06-09",
    duration == "13+" ~ "13-",
    TRUE ~ duration
  ) %>% as.ordered()
)

# Standardize post_level_premium_structure
df_raw <- df_raw %>% mutate(
  post_level_premium_structure =
    if_else(
      post_level_premium_structure == "1. Premium Jump to ART",
      "ART", "Other"
    ) %>% factor(levels = c("ART", "Other"))
)
```

^{*1} <https://github.com/kevinykuo/insurance/tree/master/data> Available for download.

```

# Format premium_jump_ratio bands
df_raw <- df_raw %>% mutate(
  premium_jump_ratio =
    case_when(
      premium_jump_ratio == "Y. Unknown" ~ "Unknown",
      premium_jump_ratio == "X. 24.01 AND UP" ~ "24.01-",
      .default = as.integer(substr(premium_jump_ratio, 4, 5)) %>%
        (function(x) sprintf("%#2d.01-%#2d.00", x, x+1))) %>%
      as.factor())

# Format categorical factor levels
levels_risk_class <-
  c("Super-Pref NS", "Pref Best NS", "Pref Resid NS", "Non-Pref NS", "Undiff NS",
    "Pref SM", "Non-Pref SM", "Undiff SM", "Agg/Unknown")
levels_premium_mode <-
  c("Annual", "Semiannual", "Quarterly", "Monthly", "Biweekly", "Unknown/Other")

df_raw <- df_raw %>% mutate(
  premium_mode = premium_mode %>% substring(4) %>%
    factor(levels = levels_premium_mode),
  risk_class = risk_class %>%
    factor(levels = levels_risk_class),
  gender = gender %>%
    factor(levels = c("M", "F"),
          labels = c("male", "female")),
  issue_age = as.ordered(issue_age),
  face_amount = face_amount %>%
    factor(levels = c("A. < 100k", "B. 100k-249k", "C. 250k-999k", "D. 1M +"),
          labels = c("-100K", "100K-250K", "250K-1M", "1M-"),
          ordered = TRUE))

# Drop unused columns and filter out zero-exposure records
df_raw <- df_raw %>% select(
  ID, lapse_study_year, policy_year,
  duration,
  issue_year, issue_age, gender, face_amount, risk_class,
  premium_mode, premium_jump_ratio, post_level_premium_structure,
  exposure_count, lapse_count,
  exposure_amount, lapse_amount
)

df_all <- df_raw %>% filter(exposure_count != 0)
df_all <- df_all %>% select(-lapse_amount) %>%
  rename(exposure = exposure_count) %>%
  rename(lapse = lapse_count)

# Calculate the observed lapse rate as the target variable
df_all <- df_all %>% mutate(lapse_rate = lapse / exposure)

prep_toint_pjratio <- function(df_raw){

```

```

df_raw %>%
  mutate(premium_jump_ratio_unknown =
    if_else(premium_jump_ratio == "Unknown", 1, 0)) %>%
  mutate(premium_jump_ratio =
    fct_recode(premium_jump_ratio,
      "05" = "Unknown", "27" = "24.01-") %>%
    substr(1, 2) %>% as.integer())
}

```

As discussed later, functional decomposition via MID is conducted on the model trained on the 2004–2006 policy years (validation year 2007). We extract and prepare this specific training and test split.

```

year_midr <- 2006 # Validation Year - 1
df_train_2006 <- df_all %>%
  filter(lapse_study_year >= year_midr-2, lapse_study_year <= year_midr)
df_test_2006 <- df_all %>%
  filter(lapse_study_year == year_midr+1)

```

1.3 Evaluation Metric Functions

We define a helper function to compute the weighted Log-loss. Setting the argument `mean = FALSE` returns the total weighted Log-loss, while `mean = TRUE` computes the weighted average.

```

xlogy <- function(x, y){
  if_else(x == 0, 0, x * log(y))
}
calc_logloss <- function(y_pred, y_act, w = NULL, mean = T){
  losses <- - xlogy(y_act, y_pred) - xlogy(1 - y_act, 1 - y_pred)
  if(is.null(w)){
    if(mean){
      return(mean(losses))
    }else{
      return(sum(losses))
    }
  }
  if(mean){
    return(sum(w * losses) / sum(w))
  }else{
    return(sum(w * losses))
  }
}

```

Next, we implement a function to compute the weighted ROC-AUC. Reference: [ROCR/R/prediction.R](#) at master ·ipa-tys/ROCR

```

calc_auc <- function(y_pred, y_act, w = NULL, verbose = FALSE){
  predictions <- y_pred
  pred.order <- order(predictions, decreasing=TRUE)
  predictions.sorted <- predictions[pred.order]
  tp <- cumsum(y_act[pred.order] * w[pred.order])
}

```

```

fp <- cumsum((1-y_act[pred.order]) * w[pred.order])
dups <- rev(duplicated(rev(predictions.sorted)))

tp <- c(0, tp[!dups])
fp <- c(0, fp[!dups])
cutoffs <- c(Inf, predictions.sorted[!dups])

n.pos <- sum(y_act* w)
n.neg <- sum((1-y_act) * w)
fn <- n.pos - tp
tn <- n.neg - fp
n.pos.pred <-
n.neg.pred <-

pred <- new("prediction", predictions=list(predictions),
          labels=list(c("0", "1")),
          cutoffs=list( cutoffs ),
          fp=list( fp ),
          tp=list( tp ),
          fn=list( fn ),
          tn=list( tn ),
          n.pos=list( n.pos),
          n.neg=list( n.neg),
          n.pos.pred=list(tp + fp),
          n.neg.pred=list(tn + fn))
auc <- performance(pred, "auc")
if (verbose){
  perf <- performance(pred, "tpr", "fpr")
  list(auc = auc, curve = perf)
}else{
  as.numeric(auc@y.values)
}
}

```

2 Model Construction

2.1 Reduced Baseline GLM

Because this study examines multiple models with distinct feature engineering requirements, implementing preprocessing workflows as model-specific functions improves code modularity and readability.

First, we construct a function to map categorical `issue_age` groups to numeric representative ages, referencing Page 39 of the SOA report.

```

prep_toint_age <- function(df_raw){
  df_raw %>%
    mutate(issue_age = case_match(issue_age,
      "0-19" ~ 9.5,
      "20-29" ~ 26.3,
      "30-39" ~ 34.7,
      "40-49" ~ 44.2,

```

```

    "50-59" ~ 53.7,
    "60-69" ~ 63.3,
    "70+" ~ 72.3
  ))
}

```

Next, we define the preprocessing function matching the SOA baseline GLM specification. Here, we construct the Reduced Baseline GLM, in which `face_amount` is intentionally omitted.

```

prepfun_soared_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
  mutate(
    issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
    risk_class_group = risk_class %>%
      factor(levels = levels_risk_class,
             labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
    premium_mode_group = premium_mode %>%
      factor(levels = levels_premium_mode,
             labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
                        "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
    premium_jump_ratio_group = cut(
      if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
      breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE)) %>%
  select(duration, post_level_premium_structure,
         issue_age, issue_age_sq, issue_age_log, risk_class_group,
         premium_mode_group, premium_jump_ratio_group, exposure, lapse)
}

```

We apply these preprocessing functions to prepare the training and test datasets.

```

df_train_2006_soared <- df_train_2006 %>% prepfun_soared_xy
df_test_2006_soared <- df_test_2006 %>% prepfun_soared_xy

```

Finally, we train the model on the preprocessed training data using `glm()`. The interaction syntax `duration : post_level_premium_structure` specifies a model stratified across these factors.

Specifying `duration : post_level_premium_structure` creates an extensive design matrix, requiring more computational time than fitting separate GLMs for each stratum. For computational efficiency, the empirical evaluations in the original paper were conducted using separately fitted GLMs across strata. While mathematically equivalent, minor numerical differences in predicted probabilities occur in strata with zero observed lapses. Consequently, the streamlined implementation in this tutorial does not strictly reproduce the paper's figures down to the last decimal place.

```

model_soared_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age - 1) : duration :
  post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_soared)

```

Policy groups absent in the training data may occasionally appear in the test data. To ensure valid predictions, preprocessing rules must be defined for the test data to handle unobserved factor levels gracefully. We implement the adjustments required for this specific evaluation year as a dedicated function.

```

prepfun_soared_xy_test_2006 <- function(df_tmp){
  df_tmp %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "10" &
      premium_jump_ratio_group == "[20,99]", "[16,20]",
      premium_jump_ratio_group)
  ) %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "12" &
      premium_jump_ratio_group == "[16,20]", "[12,16]",
      premium_jump_ratio_group)
  ) %>% mutate(
    premium_jump_ratio_group = if_else(
      post_level_premium_structure == "ART" & duration == "13-" &
      premium_jump_ratio_group == "[12,16]", "[10,12]",
      premium_jump_ratio_group)
  ) %>% mutate(
    risk_class_group = if_else(
      post_level_premium_structure == "Other" & duration == "11" &
      risk_class_group == "Super-Pref NS", "NS",
      risk_class_group)
  )
}

```

2.2 XGBoost Model

Next, we train an XGBoost predictive model as the high-performing black-box benchmark for refining the GLM.

Before feeding data into XGBoost, features must be encoded into a numeric design matrix and converted using `xgb.DMatrix()`. Additionally, columns with constant values across all rows must be removed to prevent training errors.

```

get_mm <- function(df){
  df <- df %>%
    select(setdiff(names(df),c(
      "ID", "lapse_study_year", "policy_year", "issue_year",
      "exposure_amount", "lapse", "exposure", "lapse_rate")))) %>%
    prep_toint_pjratio
  model.matrix(~ ., df)
}

df_train_2006_mm <- df_train_2006 %>% get_mm
constant_cols_2006 <- which(apply(df_train_2006_mm, 2, function(col) all(col == col[1])))
dm_train_2006 <- xgb.DMatrix(
  data = df_train_2006_mm[, -constant_cols_2006, drop = FALSE],

```

```
label = df_train_2006 %>% pull(lapse_rate),
weight = df_train_2006 %>% pull(exposure))
```

The configuration passed to `xgb.cv()` and `xgb.train()` is specified as follows. Setting `objective = "reg:logistic"` specifies a logistic loss function that outputs predicted probabilities. To prevent overfitting, `eval_metric = "logloss"` is used as the cross-validation stopping criterion. Other hyperparameters were tuned in advance using Bayesian optimization via the `tune` package ^{*2}.

```
params_xgb <- list(
  objective = "reg:logistic",
  eval_metric = "logloss", #Used in CV
  # Hyper parameters. Determined by Bayesian optimization using the "tune" package.
  gamma = 2.966439e-08, #loss_reduction
  eta = 0.02, #learn_rate
  max_depth = 10, #tree_depth
  min_child_weight = 10, #min_n
  subsample = 0.9976594, #sample_size
  colsample_bynode = 6/31 #mtry / count of predictors
)
```

With these preparations, the optimal number of boosting rounds (`nrounds`) is determined via cross-validation using `xgb.cv()`, and the final prediction model is trained using `xgb.train()`.

```
set.seed(2025)
# Determine the optimal number of boosting rounds by CV
cv_xgboost_2006 <- xgb.cv(data = dm_train_2006, nrounds = 2000, nfold = 4,
  early_stopping_rounds = 10, params = params_xgb, verbose = 0)
nrounds <- cv_xgboost_2006$early_stop$best_iteration

# Fit model
set.seed(2025)
model_xgboost_2006 <- xgb.train(data = dm_train_2006, nrounds = nrounds,
  params = params_xgb)
```

A formal comparison confirming that XGBoost outperforms the baseline GLM is provided in Section 3.

2.3 Baseline GLM / Improved GLM

Using MID, we analyze the predictions of the trained XGBoost model and use the resulting insights to systematically refine the GLM. Hereafter, the Reduced Baseline GLM constructed in Section 2.1—with `face_amount` intentionally excluded—serves as our starting point.

The following sections illustrate representative refinement steps alongside executable code examples.

^{*2} The learning rate was fixed at 0.02. The other hyperparameters were tuned once using four-fold cross-validation on the 2005–2007 training data and then held fixed across rolling windows, while the number of boosting rounds was selected separately for each window by four-fold cross-validation within the corresponding training data.

2.3.1 Functional Decomposition and Feature Importance

We prepare the dataset for performing functional decomposition. While multiple models are trained across different time windows in this study, MID inspection is conducted on the model trained on the 2004–2006 policy years. This window is selected because the prediction gap between GLM and XGBoost is sufficiently pronounced without yet being confounded by extreme market shifts ^{*3}.

We define the exposure weights and a custom prediction function (`pred.fun`) tailored to XGBoost, both of which are required by the `midr` package.

```
df_train_2006_xw <- df_train_2006 %>%
  select(-ID, -lapse_study_year, -policy_year, -issue_year, -exposure_amount, -lapse,
  ↪ -lapse_rate)
df_train_2006_w <- df_train_2006_xw %>% pull(exposure)
predfun_xgb <- function(model, newdata){
  df_mm <- newdata %>% get_mm
  df_mm <- df_mm[, -constant_cols_2006, drop = FALSE]
  predict(model, df_mm)
}
```

As the first step, we execute `midr::interpret()` to perform functional decomposition on XGBoost predictions ^{*4}. While MID of order 2 computes up to two-way interactions, we omit a subset of interactions to prevent singular matrix fits in the decomposition step.

```
mid_xgb <- midr::interpret(
  formula(
    ~ (. -exposure)*
      (duration+post_level_premium_structure+
        face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw, weights = df_train_2006_w,
  model = model_xgboost_2006, pred.fun = predfun_xgb, link = "logit",
  # Penalty factor for pseudo-smoothing; helps avoid singular fits.
  lambda = 0.01,
  verbosity = 0 # Stop when a singular fit is encountered.
)
```

Before plotting, we configure custom ggplot themes and utility functions to enhance chart aesthetics. These settings are optional for analytical execution.

```
# Display border lines.
theme_bw16 <- theme_bw(base_size = 16)
# Display the text on the X-axis vertically.
theme_axisxvert <- theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
theme_set(theme_bw16 + theme_axisxvert)
# Set default color theme (ggmid's argument "theme").
```

^{*3} As discussed in the original paper, experience variances increase substantially in magnitude across all models from the 2008 policy year onward, reflecting heightened forecasting difficulty during the financial crisis.

^{*4} For implementation details of the `midr` package, see <https://cran.r-project.org/web/packages/midr/readme/README.html>.

```

options(midr.sequential = "Oslo")
# Modify appearance of scatter plots.
update_geom_defaults("point",list(alpha = 0.5, size = 2.0, shape = 16))
# Utility function to add labels
add_label <- function(data){
  geom_label(
    data = data, aes(x = x, y = y, label = label),
    size = 5, fill = "white", colour = "black",
    label.size = 0.1, label.padding = unit(0.1, "lines"), fontface = "bold"
  )
}

```

Next, we visualize the importance of feature main effects and two-way interactions. The feature effect importance heatmap is shown in Figure 1.

```

mid_xgb %>% midr::mid.importance(
  data = df_train_2006_xw, weights = df_train_2006_w) %>%
  midr::ggmid(type = "heatmap")

```

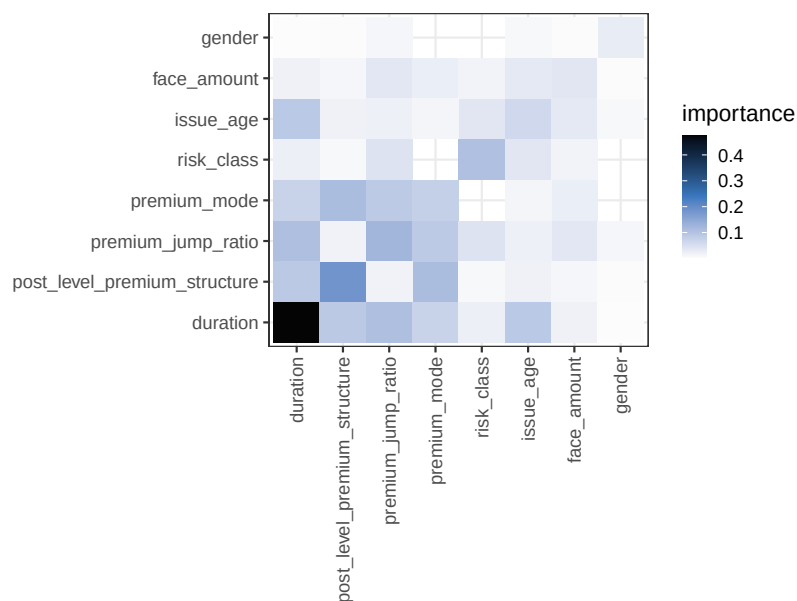


Figure1: Feature Effect Importance

By changing the decomposition target to the prediction difference between the reduced baseline GLM and XGBoost, we can directly visualize which features drive the discrepancies between the two models. The resulting importance plot for the difference model is shown in Figure 2.

Note that the decomposition is applied to the linear predictor scale (logit-transformed values) rather than raw probability predictions.

```

fn_logit <- function(p){log(p/(1-p))} # link function
fn_expit <- function(x){1/(1+exp(-x))} # inverse of the link function

# pred.fun for the SOA GLM model
predfun_soared_2006 <- function(model, newdata){
  predict(model,
    newdata %>% mutate(lapse = 0) %>% #prevent error in prepfun_soared_xy
    prepfun_soared_xy %>% prepfun_soared_xy_test_2006,
    type = c("response"))
}

# pred.fun for "difference" models
predfun_diff <- function(model, newdata){
  p_target <- model$predfun_glm(model$model_glm, newdata)
  p_base <- model$predfun_xgb(model$model_xgb, newdata)
  fn_expit(fn_logit(p_target) - fn_logit(p_base)) #apply the link function
}

# Create the "difference" model
model_diff <- list()
model_diff$model_xgb <- model_xgboost_2006
model_diff$model_glm <- model_soared_2006
model_diff$predfun_xgb <- prepfun_xgb
model_diff$predfun_glm <- prepfun_soared_2006

# Execute midr::interpret for the "difference" model
mid_diff <- midr::interpret(
  formula(
    ~ (. -exposure)*
      (duration+post_level_premium_structure+face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw, weights = df_train_2006_w,
  model = model_diff, pred.fun = prepfun_diff,
  link = "logit", lambda = 0.01,
  verbosity = 0
)

# Label settings
add_label_tmp <- add_label(data.frame(
  x = c("post_level_premium_structure",
    "risk_class", "premium_mode", "premium_mode", "premium_mode",
    "issue_age", "risk_class", "premium_jump_ratio", "issue_age",
    ↪ "premium_jump_ratio", "gender"),
  y = c("post_level_premium_structure",
    "risk_class", "premium_mode", "premium_jump_ratio",
    ↪ "post_level_premium_structure",
    "issue_age", "issue_age", "risk_class", "face_amount", "face_amount", "gender"),
  label = c("X", "A-2", "A-3", "A-3", "A-4", "A-5", "A-6", "A-7", "A-8", "A-9", "A-10")
))

```

```
#display feature effect imporatance plot
mid_diff %>% midr::mid.importance(
  data = df_train_2006_xw, weights = df_train_2006_w) %>%
  midr::ggmid(type = "heatmap") + add_label_tmp
```

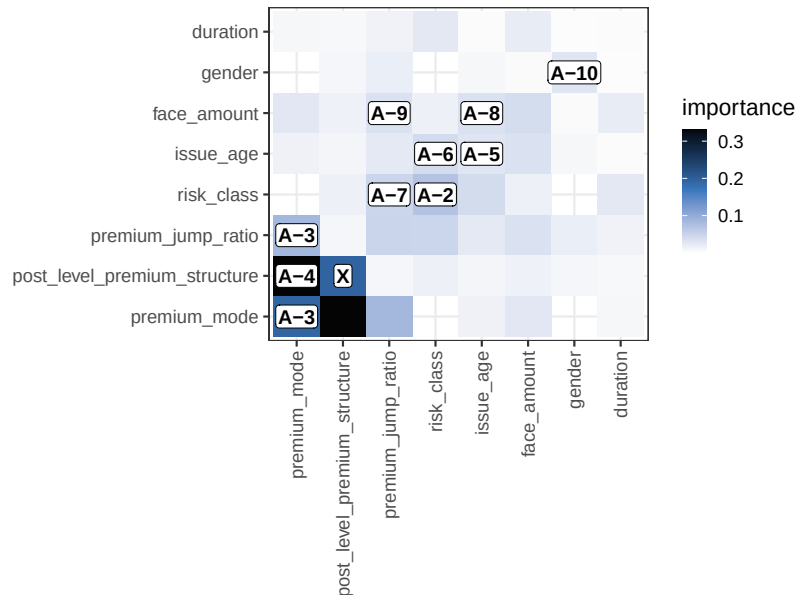


Figure2: Feature Effect Importance of the difference model

Because the GLM is stratified, focused analysis on specific strata can also be conducted. For example, restricting the decomposition to the `duration == 10` and `post_level_premium_structure == "ART"` stratum yields Figure 3.

```
# Create filtered data.
df_train_2006_xw_d10ART <- df_train_2006_xw %>%
  filter(duration == 10, post_level_premium_structure == "ART")
df_train_2006_w_d10ART <- df_train_2006_xw_d10ART %>% pull(exposure)

# midr::interpret.
mid_xgb_d10ART <- midr::interpret(
  formula(
    ~ (. -exposure - duration - post_level_premium_structure)*
      (face_amount+premium_jump_ratio+issue_age)),
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART,
  model = model_xgboost_2006, pred.fun = predfun_xgb,
  link = "logit", lambda = 0.01, verbosity = 0
)
mid_diff_d10ART <- midr::interpret(
  formula(
    ~ (. -exposure - duration - post_level_premium_structure)*
      (face_amount+premium_jump_ratio+issue_age)),
```

```

    data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART,
    model = model_diff, pred.fun = predfun_diff,
    link = "logit", lambda = 0.01, verbosity = 0
  )

# Display feature effect importance plot.
mid_xgb_d10ART %>% midr::mid.importance(
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART) %>%
  midr::ggmid(type = "heatmap") +
  ggtitle("XGBoost")

# Label settings
add_label_tmp <- add_label(data.frame(
  x = c("premium_jump_ratio", "face_amount", "risk_class", "premium_mode", "risk_class"),
  y = c("premium_jump_ratio", "face_amount", "risk_class", "premium_jump_ratio",
        "premium_jump_ratio"),
  label = c("Y", "B-1", "B-2", "B-3", "B-7")
))

# Display feature effect importance plot.
mid_diff_d10ART %>% midr::mid.importance(
  data = df_train_2006_xw_d10ART, weights = df_train_2006_w_d10ART) %>%
  midr::ggmid(type = "heatmap") +
  ggtitle("GLM - XGBoost") + add_label_tmp

```

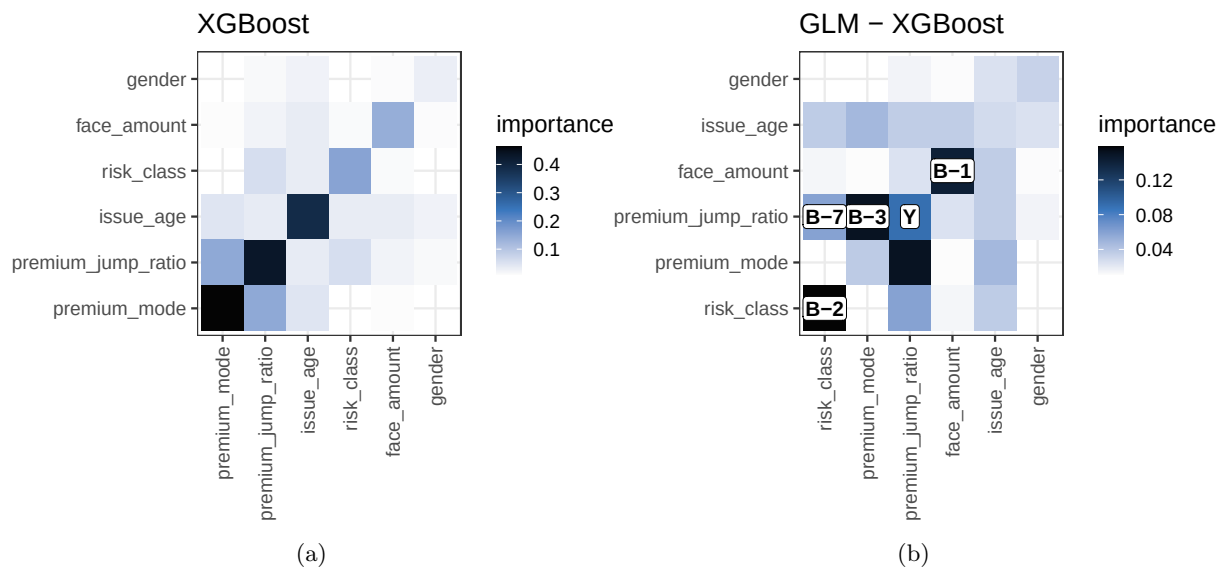


Figure3: Feature Effect Importance in Duration = 10, ART

2.3.2 Model Refinements

We examine the difference importance plots (Figure 2 and Figure 3b) and focus our analysis on features showing large prediction discrepancies.

Not all discrepancies lead directly to actionable GLM modifications. For instance, point X in Figure 2

indicates a discrepancy in `post_level_premium_structure`, but since the GLM is already stratified by this factor, main effect modifications cannot be explicitly added. Similarly, point Y in Figure 3b indicates a discrepancy in `premium_jump_ratio`, which is already binned into dummy variables in the reduced baseline GLM.

Excluding these cases, we evaluate the high-discrepancy components in order of magnitude.

In this tutorial, we detail the refinement workflow for the first three items (I-1 to I-3). The remaining items (I-4 to I-10) are omitted here for brevity.

I-1: Main Effect of `face_amount`

Point B-1 in Figure 3b highlights a prominent discrepancy in the main effect of `face_amount`. While omitted in our reduced baseline GLM, MID indicates its strong influence on lapse predictions.

To plot the function terms obtained from functional decomposition, we use the `midr::ggmid()` function.

Plotting the main effect function term of `face_amount` for `duration == 10`, ART (Figure 4) reveals that higher face amounts correspond to higher predicted lapse rates.

```
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw, term = "face_amount")
```

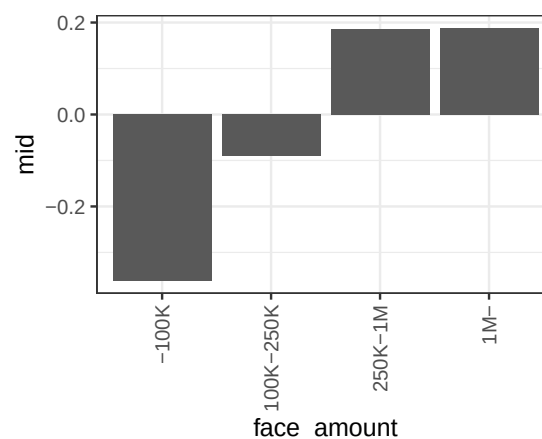


Figure4: Main Effect Plot of `face_amount`

To examine whether this relationship holds across other durations, Figure 5 displays the compound effect (main effects + interaction) of `face_amount` and `duration`.

```
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
  term = "face_amount:duration", main.effects = TRUE,
  type = "compound") #add sample points to the plot
```

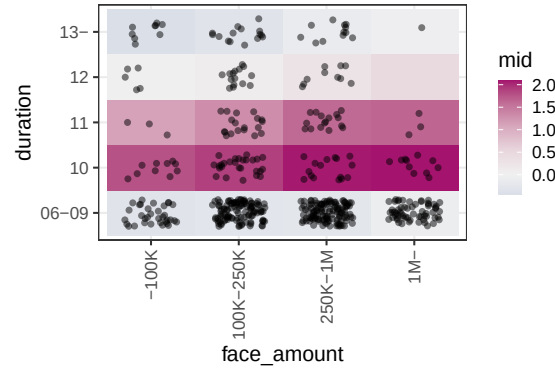


Figure5: Main Effect + Interaction Plot of duration and face_amount

Figure 5 indicates that for early durations ($\text{duration} \leq 9$), higher face amounts correlate with lower lapse rates, whereas the relationship reverses at duration 10 and beyond. Adding `face_amount` as a predictor back into the GLM is expected to improve predictive performance.

I-2: Main Effect of risk_class

```
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                        term = "risk_class:duration", main.effects = TRUE,
                        type = "compound")
mid_xgb %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                        term = "premium_mode:duration", main.effects = TRUE,
                        type = "compound")
```

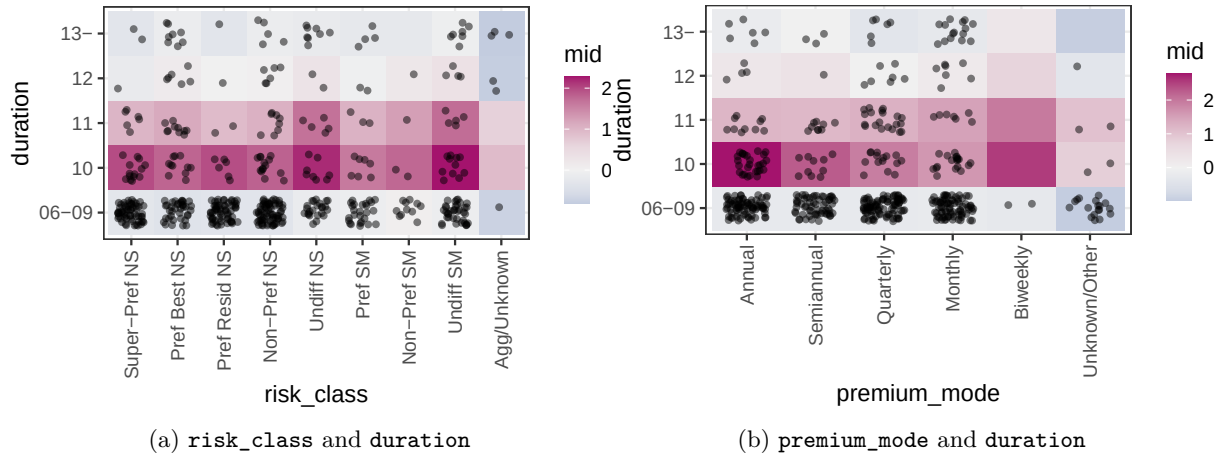


Figure6: Main Effects + Interaction Plots

Points A-2 (Figure 2) and B-2 (Figure 3b) highlight a large discrepancy in `risk_class`. Plotting the compound effect of `risk_class` and `duration` (Figure 6a) reveals that while the baseline GLM consolidated risk classes into three broad categories, undifferentiated groups ("Undiff NS", "Undiff SM") and "Agg/Unknown" exhibit distinct lapse trends. Therefore, we update the GLM to treat "Undiff

"NS", "Undiff SM", and "Agg/Unknown" as independent factor levels ^{*5}.

I-3: Interaction between premium_mode and premium_jump_ratio

Points A-3 and B-3 indicate a significant discrepancy in the interaction between `premium_mode` and `premium_jump_ratio`. Examining the main effect of `premium_mode` across durations (Figure 6b) shows clear differences between "Semiannual" and "Quarterly" modes, prompting us to separate them into distinct levels in the GLM.

Next, to examine the interaction between `premium_mode` and `premium_jump_ratio`, Figure 7 plots their combined effects at `duration == 10`, ART. Specifically, Figure 7a displays both the main effects and the interaction term, whereas Figure 7b isolates the interaction effect alone ^{*6}.

Whether to add the main effects to the interaction plot can be controlled via the `main.effects` argument in `ggmid()`.

```
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                               term = "premium_jump_ratio:premium_mode", main.effects = TRUE,
                               type = "compound") +
  ggtitle("Duration = 10, ART, Main Effects + Interaction")
mid_xgb_d10ART %>% midr::ggmid(data = df_train_2006_xw %>% sample_n(500),
                               term = "premium_jump_ratio:premium_mode",
                               type = "compound") +
  ggtitle("Duration = 10, ART, Interaction Only")
```

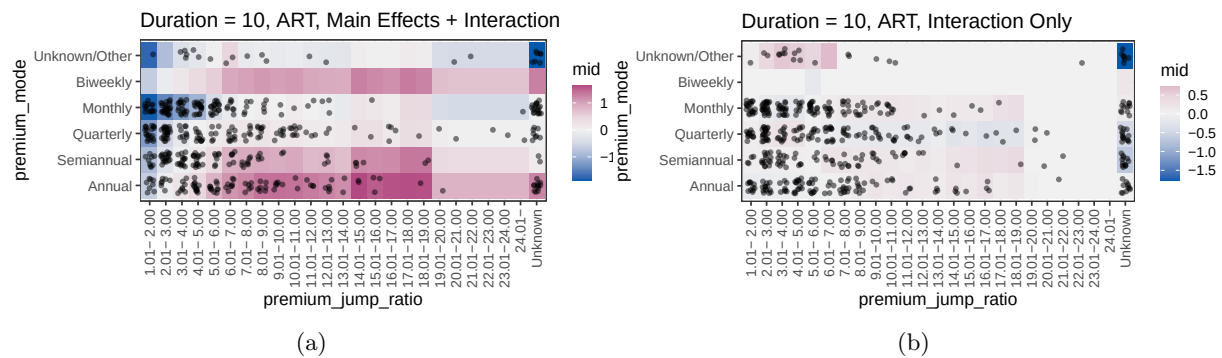


Figure7: Effects of `premium_jump_ratio` and `premium_mode`

As indicated in Figure 7b, for the "Quarterly" mode, the relationship where higher `premium_jump_ratio` leads to higher lapse rates is substantially weakened. Therefore, we adopt a modification adding an interaction term between `premium_mode` and `premium_jump_ratio` exclusively when `duration == 10` and `premium_mode == "Quarterly"`.

^{*5} To avoid overfitting in small strata, collapsing rules are applied based on sample counts: (i) Merge "Super-Pref NS" with "NS" if count < 50; (ii) Merge "Undiff NS" and "NS" if either count < 700; (iii) Merge "Agg/Unknown" with "Undiff SM" if count < 100; (iv) Merge "SM" with "Undiff SM" if count < 700.

^{*6} To prevent excessive model complexity, coarse binning (`breaks = c(0,1,2,5,99)`) is applied to `premium_jump_ratio` for this specific interaction term. If a prediction is required for a bin that lacks observed data in the training set, it is remapped to the nearest bin containing observed data, prioritizing a higher bin if available, or a lower bin otherwise.

2.3.3 Fitting Refined Models

2.3.3.1 Fitting Baseline GLM

We construct the Baseline GLM—which restores `face_amount` per modification Item I-1 and replicates the original SOA model—by applying minor modifications to the preprocessing function.

```
prepfun_soa_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
    mutate(
      issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
      risk_class_group = risk_class %>%
        factor(levels = levels_risk_class,
              labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
      premium_mode_group = premium_mode %>%
        factor(levels = levels_premium_mode,
              labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
                        "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
      premium_jump_ratio_group = cut(
        if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
        breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE)) %>%
    select(duration, post_level_premium_structure,
           issue_age, issue_age_sq, issue_age_log, risk_class_group,
           premium_mode_group, premium_jump_ratio_group,
           face_amount, #I-1
           exposure, lapse)
}
```

```
df_train_2006_soa <- df_train_2006 %>% prepfun_soa_xy
df_test_2006_soa <- df_test_2006 %>% prepfun_soa_xy

model_soa_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age - 1) : duration :
  ↪ post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_soa)
```

2.3.3.2 Fitting Improved GLM

The Improved GLM incorporating all ten refinement items identified in the original paper is constructed as follows. Most enhancements are embedded directly into the preprocessing pipeline.

```
prepfun_mod_xy <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
    mutate(
      premium_mode_oo = #I-4
      if_else(
        duration >= "10" & post_level_premium_structure == "Other" &
        premium_mode == "Unknown/Other", 1, 0),
```

```

post_level_premium_structure = if_else( #I-4
  premium_mode_oo == 1, "ART", post_level_premium_structure),
premium_mode = if_else( #I-4
  premium_mode_oo == 1, "Monthly", premium_mode),
issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
issue_age_70 = if_else(issue_age >= 70 & duration == "06-09", 1, 0), #I-5
issue_age_d10ART = if_else( #I-8
  (duration == "06-09" | duration == "10") & post_level_premium_structure == "ART",
  issue_age, 0
),
# risk_class_group = risk_class %>% #cf) non #I-2
# factor(levels = levels_risk_class,
#   labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
risk_class_group = risk_class %>% #I-2
  factor(levels = levels_risk_class,
    labels = c("Super-Pref NS", "NS", "NS", "NS", "Undiff NS",
      "SM", "SM", "Undiff SM", "Agg/Unknown")),
risk_class_group = if_else( #I-2 (adjusted to 2006-train)
  risk_class_group == "Undiff NS" &
    (duration != "06-09" | post_level_premium_structure != "ART"),
  "NS",
  if_else(
    risk_class_group == "Undiff SM" &
      (duration >= "11" & post_level_premium_structure == "ART" |
        duration <= "12" & post_level_premium_structure == "Other") |
    risk_class_group == "Agg/Unknown" &
      (duration <= "11" & post_level_premium_structure == "ART"),
    "SM",
    risk_class_group
  )
),
risk_class_int = factor( #I-6 (adjusted to 2006-train)
  if_else(
    (risk_class == "Undiff SM" | risk_class == "Non-Pref NS") & duration == "06-09" |
    (risk_class == "Non-Pref SM" | risk_class == "Undiff NS" ) &
    duration == "06-09" & post_level_premium_structure == "ART" |
    risk_class == "Non-Pref NS" &
    duration == "10" & post_level_premium_structure == "ART",
    risk_class, "Super-Pref NS"),
  levels = levels_risk_class),
premium_mode_group = premium_mode %>% #cf) non #I-3
  factor(levels = levels_premium_mode,
    labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
      "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_mode_group = premium_mode %>% #I-3
  factor(levels = levels_premium_mode,
    labels = c("Annual", "Semiannual", "Quarterly",
      "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_mode_group_q = if_else( #I-3
  duration == "10" & premium_mode == "Quarterly", 1, 0),

```

```

premium_jump_ratio_group = cut(
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE),
premium_jump_ratio_group2 = cut( #I-3
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 5, 99), right = FALSE),
premium_jump_ratio_rc_NPNS = if_else( #I-7 (adjusted to 2006-train)
  !(duration >= "11" & post_level_premium_structure == "Other") &
  risk_class == "Non-Pref NS", premium_jump_ratio, 0),
premium_jump_ratio_rc_PBNS = if_else( #I-7 (adjusted to 2006-train)
  !(duration == "12" & post_level_premium_structure == "Other") &
  risk_class == "Pref Best NS" & (duration >= "10"), premium_jump_ratio, 0),
face_amount_d09PrUnHi = if_else( #I-9 (adjusted to 2006-train)
  duration == "06-09" & post_level_premium_structure == "ART" &
  premium_jump_ratio_unknown == 1 & face_amount == "1M-",
  1, 0)
) %>%
select(duration, post_level_premium_structure,
  issue_age, issue_age_sq, issue_age_log, risk_class_group,
  issue_age_70, #I-5
  issue_age_d10ART, #I-8
  risk_class_int, #I-6
  premium_mode_group, premium_jump_ratio_group,
  premium_mode_oo, #I-4
  premium_jump_ratio_group2, premium_mode_group_q, #I-3
  premium_jump_ratio_rc_NPNS, premium_jump_ratio_rc_PBNS, #I-7
  face_amount, #I-1
  face_amount_d09PrUnHi, #I-9
  gender, #I-10
  exposure, lapse)
}

```

In the model-fitting step, only targeted interaction terms need to be explicitly specified in the formula; newly engineered features are automatically included via the dot (.) syntax.

```

df_train_2006_mod <- df_train_2006 %>% prepfun_mod_xy
df_test_2006_mod <- df_test_2006 %>% prepfun_mod_xy

model_mod_2006 <- glm(
  cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age
    -premium_jump_ratio_group2 -premium_mode_group_q #I-3
    + premium_jump_ratio_group2 : premium_mode_group_q #I-3
    + risk_class_int * issue_age #I-6
    + issue_age_d10ART : face_amount #I-8
    - 1) : duration : post_level_premium_structure,
  family = quasibinomial(),
  data = df_train_2006_mod)

```

3 Statistical Predictive Performance

3.1 Reduced Baseline GLM

The evaluation metrics for the Reduced Baseline GLM are calculated as follows. Passing `mean = FALSE` to `calc_logloss()` outputs total weighted Log-loss, while `mean = TRUE` outputs the weighted average.

```
pred_soared <- model_soared_2006 %>%  
  predict(df_test_2006_soared %>% prepfun_soared_xy_test_2006, type = c("response"))  
  
calc_logloss(  
  pred_soared,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = TRUE  
)#0.2988156
```

[1] 0.2988156

```
calc_logloss(  
  pred_soared,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),  
  mean = FALSE  
)#238153.1
```

[1] 238153.1

```
calc_auc(  
  pred_soared,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure)  
)#0.8065347
```

[1] 0.8065347

3.2 XGBoost Model

The evaluation metrics for XGBoost are shown below, successfully replicating the figures reported in the original paper (for validation year 2007).

```
df_test_2006_mm <- df_test_2006 %>% get_mm  
df_test_2006_mm <- df_test_2006_mm[, -constant_cols_2006, drop = FALSE]  
pred_xgb <- model_xgboost_2006 %>% predict(df_test_2006_mm)  
  
calc_logloss(  
  pred_xgb,  
  df_test_2006 %>% pull(lapse_rate),  
  df_test_2006 %>% pull(exposure),
```

```
    mean = TRUE
  )#0.2935507
```

[1] 0.2935507

```
calc_logloss(
  pred_xgb,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#233957.1
```

[1] 233957.1

```
calc_auc(
  pred_xgb,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.820604
```

[1] 0.820604

3.3 Baseline GLM

The evaluation metrics for the Baseline GLM are shown below. As noted in Section 2.1, a slight numerical difference in Log-loss arises relative to the paper due to single-model versus multi-strata implementation.

```
pred_soa <- model_soa_2006 %>%
  predict(df_test_2006_soa %>% prepfun_soared_xy_test_2006, type = c("response"))

calc_logloss(
  pred_soa,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)#0.2984396
```

[1] 0.2984396

```
calc_logloss(
  pred_soa,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#237853.5
```

[1] 237853.5

```
calc_auc(
  pred_soa,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.8073218
```

[1] 0.8073218

3.4 Improved GLM

The evaluation metrics for the Improved GLM are shown below. As with the baseline model, Log-loss exhibits a minor numerical shift from the paper. Consistent with the paper's findings, Log-loss shows a slight degradation relative to the Baseline GLM in this particular validation year, while AUC shows noticeable improvement.

```
pred_mod <- model_mod_2006 %>%
  predict(df_test_2006_mod %>% prepfun_soared_xy_test_2006, type = c("response"))

calc_logloss(
  pred_mod,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = TRUE
)#0.2984574
```

[1] 0.2984574

```
calc_logloss(
  pred_mod,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#237867.7
```

[1] 237867.7

```
calc_auc(
  pred_mod,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.8090849
```

[1] 0.8090849

Acknowledgments

The authors gratefully acknowledge the Society of Actuaries (SOA) for providing the public dataset (SOA 10-Year Level Premium Term Lapse Study) utilized in this study. In addition, generative AI tools (such as Gemini and ChatGPT) were used to assist with English language editing and R code refinement,

with all final content thoroughly reviewed and verified by the authors.

References

- Kozuma, R., Asashiba, R., Matsumori, Y., & Iwasawa, H. (2026). *Enhancing GLM-based lapse rate models for EV calculation using interpretable machine learning*.
- Society of Actuaries. (2014). *2014 post level term lapse & mortality report*. <https://www.soa.org/resources/experience-studies/2014/research-2014-post-level-shock/>